Collaborative Digital Art Layer Manager: Details

Supporting detail for the problem statement

1. The Core Idea

Each design layer is a node in a doubly linked list. Every node points to the layer above it (**next**) and the layer below it (**prev**). The list keeps a **head** pointer for the bottom layer and a **tail** pointer for the top layer. Stacking order is simply the order of the list.

2. What Each Layer Stores

- **name:** the label shown to the user
- **pixelRef:** a reference to the layer's pixel data, not a copy
- **opacity:** a value from 0 to 1 that controls how strongly the layer blends
- **visible:** whether the layer is drawn
- **active:** whether the layer is kept; inactive layers are deleted
- **prev** and **next:** links to neighbouring layers

3. The Four Required Operations

- **Insert at top or bottom.** A new layer is attached to the tail (top) or head (bottom). Only the new node and its one neighbour need pointer updates, so this takes constant time.
- **Delete inactive layers.** The list is scanned once. Each inactive node is removed by connecting its neighbours to each other. The scan saves the next node before unlinking, so the traversal doesn't break. Head, tail, and middle deletions follow the same pattern.
- **Reorder via pointer manipulation.** Moving a layer up or down means rewiring a few pointers around it, not copying pixel data. Moving to a specific position is done by removing the layer and reinserting it at the target spot. Boundary cases, such as moving the top layer up, must be checked first.
- **Sequential traversal to render.** Starting at the head, each visible layer is blended onto a blank canvas using its opacity, moving upward until the tail is reached. Because the drawing order matches the list order, the final image is correct.

4. Edge Cases to Handle

- Empty list (head and tail are null)
- Single layer, where head and tail are the same node
- Deleting the head, the tail, or every layer
- Moving the top layer up or the bottom layer down
- Hidden layers, which must contribute no pixels when rendered

5. Invariants That Must Always Hold

- **head.prev** and **tail.next** are null.
- For every link, if A.next = B, then B.prev = A.
- Traversing forward and backward visits the same layers in reverse order.
- A removed node is no longer reachable from the list.

6. Complexity

- Insert at either end: O(1)
- Delete a known node: O(1)
- Delete all inactive layers: O(n), one pass
- Move up or down: O(1)
- Move to an index: O(n), because the list must be scanned to reach the index
- Render: O(n) layer passes, plus the cost of blending each pixel

7. Why a Doubly Linked List

A singly linked list can't reach a node's predecessor without scanning from the head, which makes moving a layer down awkward. Arrays make middle insertion and deletion expensive because elements must shift. The doubly linked list gives fast local changes at both ends and in the middle, and it makes reordering a matter of changing a few pointers.

8. Deliverables and Success Criteria

- Source code for the list and layer node
- A driver program that demonstrates insertion, deletion, reordering, and rendering
- Test cases for the edge cases above
- A short explanation of the design and complexity
- Success means every test passes, forward and backward traversal agree, and the rendered layer order matches the list order.